Battleship - System Design Document

1. Introduction

1.1 Background

Battleship is a classic two-player strategy-based guessing game. The game is played with four 10x10 grids where each square is identified by a coordinate; one letter (A to J) and one number (1 to 10). Each player uses two grids; one for placing ships and recording enemy attacks, and another for recording the player's own attacks. Before a game begins, each player secretly arranges their fleet of ships on their primary grid. A ship occupies a select number of consecutive squares and can be oriented horizontally or vertically. The number of squares that a ship occupies is dependent on the ship type. Ships cannot overlap.

The game begins once both players have placed all of their ships. Players alternate taking turns by guessing a square on their enemy's primary grid. Guesses are identified by the coordinate corresponding to the square (ex: E7). If a player guesses the correct location of an enemy ship they are awarded a "hit". If not, the player receives a "miss". The result of a player's attack is recorded on their secondary grid. Players continue to alternate taking turns. A ship is destroyed if all squares that it occupies are hit. The first player to destroy all of their enemy's ships wins the game.

1.2 Purpose

The purpose of our project is to upgrade the classic game of Battleship. The goal is to add exciting contemporary features to the game, providing Battleship players with a new experience while still enjoying an old favourite. Users will have the opportunity to play against the computer (with 2 difficulty levels) or pass and play with a friend. This project brings creative new aspects to the game such as different maps to choose from, impressive power-ups, different fleet layouts, and animations to celebrate successful hits. The user will play the entire game through the GUI; there will be no need for a command-line inter face.

1.3 Design Goals

Performance

- **Response Time**
    - The ScreenManager must respond to input and display the correct screen within one second.
    - Opponent fleet layout must be generated on OpponentGrid within two seconds.
    - GridSquares must be updated within one second of either an attack from Player or Opponent.
    - Opponent must respond to Player within two seconds.
- **Memory**
    - The game must run on 4 GB of RAM so it can be played on any laptop.

Dependability

- **Robustness**
    - User Input is controlled by a mouse.
    - ScreenManager will show the correct screen.
- **Fault Tolerance**
    - If Player selects a grid that has been attacked, an error message will pop up within 2 seconds.
        - Once user acknowledges message, gameplay continues.

Cost

- **Development Cost**
    - The game is mainly developed using SFML and written in C++.
    - The GUI is mainly developed using Aseprite.
- **Deployment Cost**
    - The game needs to be downloaded and is run using an executable file.
- **Administration Cost**
    - The system will be administered by group members.

Maintenance

- **Extensibility**
  - Structured classes allow for additional features to be added within a day.
- **Modifiability**
  - The game is built with proper architecture and standards so that it can be updated efficiently.
- **Adaptability**
  - The game is run using an executable file associated with Player's OS (Windows or Mac).
- **Portability**
  - The game is able to run on Windows and Mac operating systems for more flexibility.
- **Readability**
  - The code will be well documented and can be understood by anyone with coding experience.
- **Traceability of Requirements**
  - Mapping the code to specific requirements can be done by referring to the specific class structure.

End User Criteria

- **Utility**
  - The system provides the Player with detailed information of the game status throughout the entirety of the game.
- **Usability**
  - The game will be executable by the Player using only the mouse.

2. System Design

2.1 General Use Cases

The following section describes the flow of events within the general PvE gameplay and PvP gameplay use cases.

PvE Gameplay Use Case

Participating Actors:

- Player
- Computer Opponent

Flow of Events:

1. Player initiates the game by pressing the "Play" button on the Homepage screen.
2. The ScreenManager responds by sending Player to the DifficultySelection screen. The screen presents two difficulty options to Player.
3. After Player has selected a difficulty the option, the ScreenManager responds by sending Player to the FleetPlacement screen. Player then arranges their fleet of Ships on PlayerGrid. The location of each Ship is updated based on its final position. Player then begins the game play by pressing the "Start" button.
4. Opponent is generated based on the difficulty specified by Player. A fleet layout is randomly generated on OpponentGrid.
5. The ScreenManager sends Player to the Gameplay screen and the game loop begins. Player starts by attacking Opponent. If the attack is successful, the associated GridSquare of OpponentGrid is updated as "hit". If the attack is unsuccessful, the associated GridSquare of OpponentGrid is updated as "missed". Opponent then responds by attacking Player. The PlayerGrid is updated accordingly. Player and Opponent continue alternating until all GridSquare's occupied by Ships on either PlayerGrid or OpponentGrid are marked as "hit".
6. The ScreenManager responds by sending Player to the Result screen. Player is then sent to the Homepage screen by pressing the "Back to Homepage" button.

Entry Conditions:

- Player initiates the game

Exit Conditions:

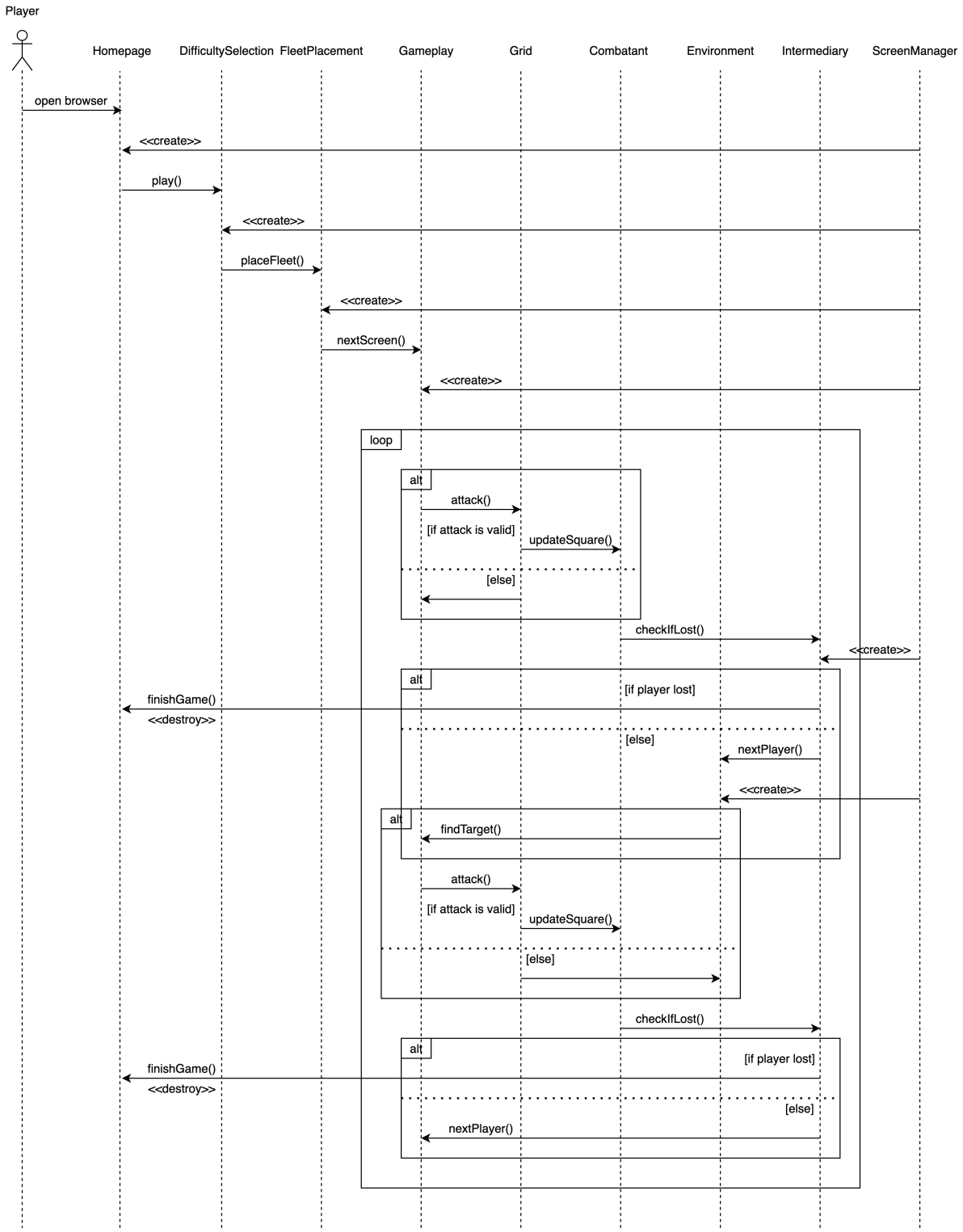
- Either Player or Opponent wins the game
- Player forfeits the game

Quality Requirements:

- The ScreenManager must respond to input and display the correct screen within one second.
- Opponent fleet layout must be generated on OpponentGrid within two seconds.
- GridSquares must be updated within one second of either an attack from Player or Opponent.
- Opponent must respond to Player within two seconds.

Figure 2 diagrammatically represents the PvE use case and displays the timeline of object creation and destruction.

Figure 2: PvE Sequence Diagram



PvP Gameplay Use Case

Participating Actors:

- Player1
- Player2

Flow of Events:

1. Player1 initiates the game by pressing the "Play" button on the Homepage screen.
2. The ScreenManager responds by sending Player1 to the FleetPlacement screen. Player1 then arranges their fleet of Ships on Player1Grid. The location of each Ship is updated based on its final position. Player1 confirms placement by pressing the "Start" button. The ScreenManager sends Player1 to the Intermediary screen. Player1 hands control to Player2.
3. Player2 initiates their fleet placement by pressing the "Continue" button. The ScreenManager responds by sending Player2 to the FleetPlacement screen. Player2 then arranges their fleet of Ships on Player2Grid. The location of each Ship is updated based on its final position. Player2 confirms placement by pressing the "Start" button. The ScreenManager sends Player2 to the Intermediary screen. Player2 hands control to Player1.
4. Player1 initiates the game loop by pressing the "Continue" button. The ScreenManager responds by sending Player1 to the Gameplay screen. Player1 starts by attacking Player2. If the attack is successful, the associated GridSquare of Player2Grid is updated as "hit". If the attack is unsuccessful, the associated GridSquare of Player2Grid is updated as "missed". The ScreenManager sends Player1 to the Intermediary screen. Player1 hands control to Player2. Player2 initiates their attack by pressing the "Continue" button. Player2 then responds by attacking Player1. The Player1Grid is updated accordingly.
5. The ScreenManager sends Player2 to the Intermediary screen. Player2 hands control to Player1. Player1 and Player2 continue alternating until all GridSquare's occupied by Ships on either Player1Grid or Player2Grid are marked as "hit".
6. The ScreenManager responds by sending either Player1 or Player2 to the Result screen. Both Player1 and Player 2 are then sent to the Homepage screen after either pressing the "Back to Homepage" button.

Entry Conditions:

- Player1 initiates the game

Exit Conditions:

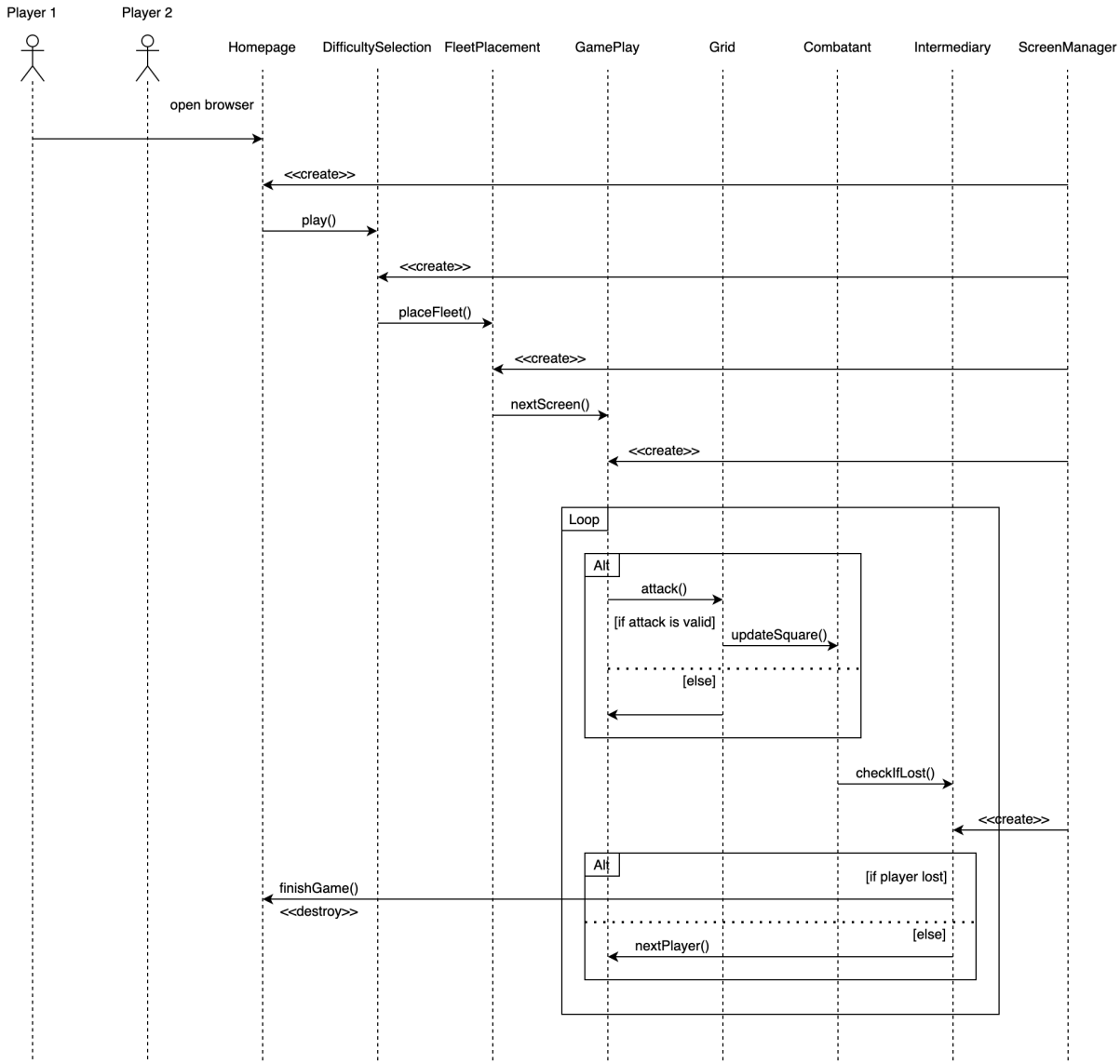
- Either Player1 or Player2 wins the game
- Either Player1 or Player2 forfeits the game

Quality Requirements:

- The ScreenManager must respond to input and display the correct screen within one second.
- GridSquares must be updated within one second of either an attack from Player1 or Player2.

Figure 3 diagrammatically represents the PvE use case and displays the timeline of object creation and destruction.

Figure 3: PvP Sequence Diagram



2.2 Boundary Use Cases

The following section describes the flow of events within the boundary startup, shutdown, and exception use cases.

Startup Use Case

Participating Actors:

- System User

Flow of Events:

1. System User will initialize startup up by running the game executable associated with their operating system (Windows or macOS).
2. Upon startup the game window will be created. The ScreenManager will create the initial game state and display the Homepage screen to the user.
3. No persistent data will be loaded on startup as the system does not store any persistent data.

Shutdown Use Case

Participating Actors:

- System User

Flow of Events:

1. System User will initialize shutdown up by closing the game window.
2. Upon shutdown all active objects will be destroyed and any memory associated with the game will be cleared or deallocated. The ScreenManager will close the game window.
3. No persistent data will be stored on shutdown as the system does not store any persistent data.

Exception Use Case

Participating Actors:

- System User

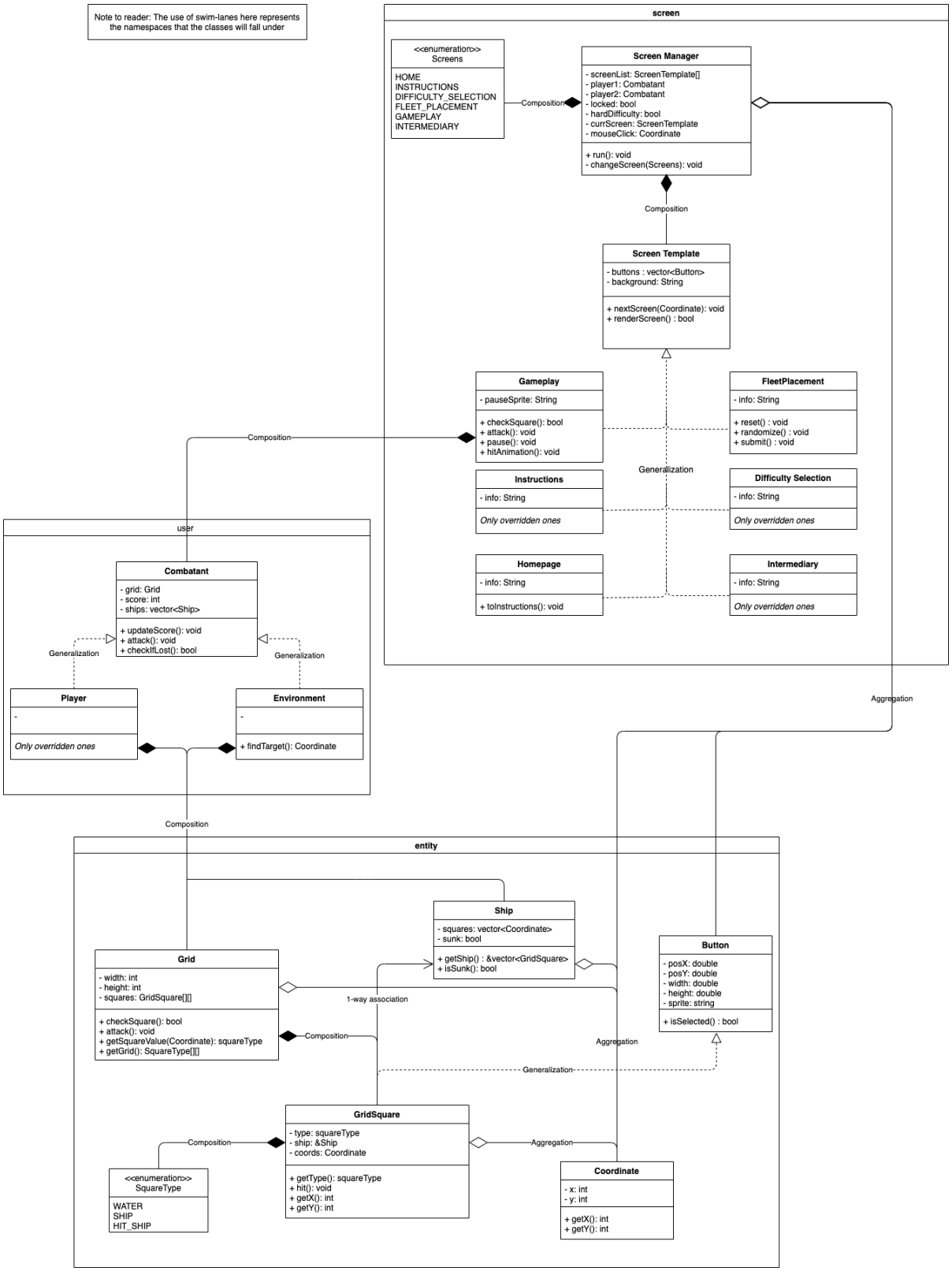
Flow of Events:

1. Exceptions may be generated by hardware failures, operating environmental changes, or software faults.
2. The system will handle exceptions by treating all exception use cases as shutdown use cases. If an exception is detected, all active objects will be destroyed and any memory associated with the game will be cleared or deallocated. The ScreenManager will close the game window.
3. No persistent data will be stored or corrupted when exception handling as the system does not store any persistent data.

2.3 Subsystem Decomposition

The following UML diagram displays the contents of each class within in the system and their relationships to one another.

Figure 4: UML Class Diagram



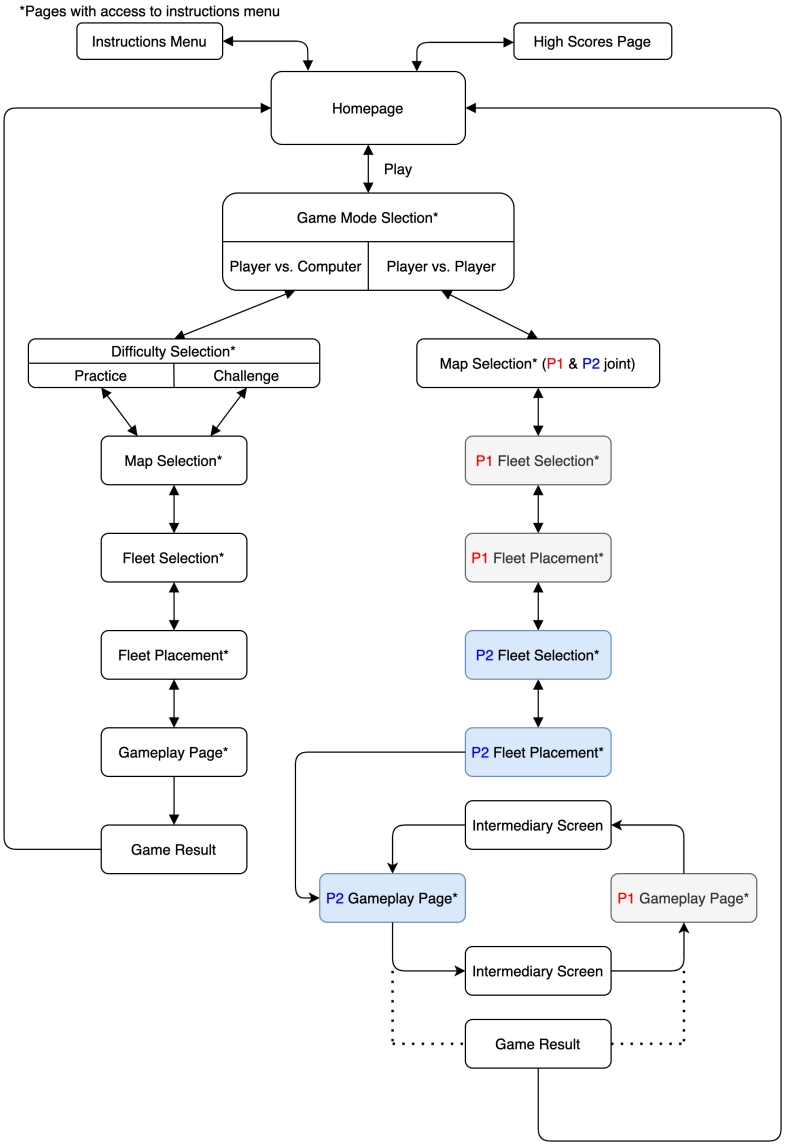
The following table classifies and describes each object presented in the preceding UML diagram.

Table 1: Object Classification

Boundary Objects	Entity Objects	Control Objects
Screen Template: Parent class for every different screen that is rendered to the application window.	**Combatant**: Parent class for the two types of players (human players and the computer).	**Screen Manager**: Top level class that manages the game loop.
Gameplay: Screen which extends the **Screen Template**. Used to display the main game screen to the user.	**Player**: Extends the **Combatant** class. Represents a human player in the game.	
Instructions: Screen which extends the **Screen Template**. Used to display the instructions to players at any time during the game.	**Environment**: Extends the **Combatant** class. Represents a computer player in the game.	
Fleet Placement: Screen which extends the **Screen Template**. Used to display the Fleet Selection and Placement Screens.	**Grid**: Class which contains a collection of Grid Squares (Buttons). Represents the grid displayed on the Gameplay screen.	
Difficulty Selection: Screen which extends the **Screen Template**. Used to display the Difficulty Selection Screen.	**Ship**: Class which contains information about the ships on the grid. Represents a ship on the Gameplay screen.	
Homepage: Screen which extends the **Screen Template**. Used to display the Homepage screen.	**Coordinate**: Helper class to represent a pair of integers which refer to a Grid Square in the Grid.	
Intermediary: Screen which extends the **Screen Template**. Used to display a message to the user to indicate that the player's turn is over and that the second players turn has begun.	**Screens**: Enumerated type for each of the different screens.	
Button: Parent class for all the buttons used in any of the screens.	**Square Type**: Enumerated type for each of the different types of Grid Squares statuses.	
Grid Square: Extends the **Button** class. Special button that represents a square on the BattleShip Grid. (clickable so the user can select for attack)		

The following state chart diagram displays all possible states of the ScreenManager object throughout a gameplay session.

Figure 1: ScreenManager State Chart Diagram



3. GUI Interfaces

The following three figures are prototypes of the homepage, fleet selection, and gameplay screens within the game.

Figure 5: Homepage

Figure 6: Fleet Selection Screen

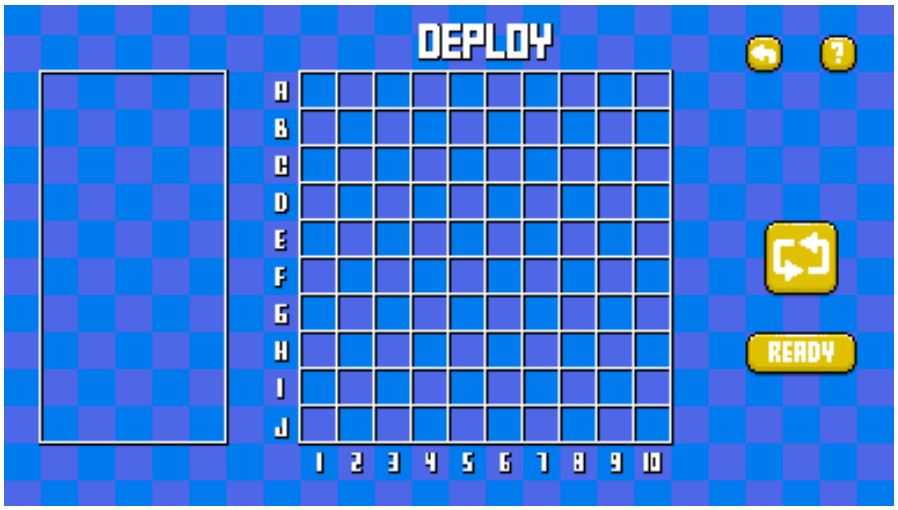
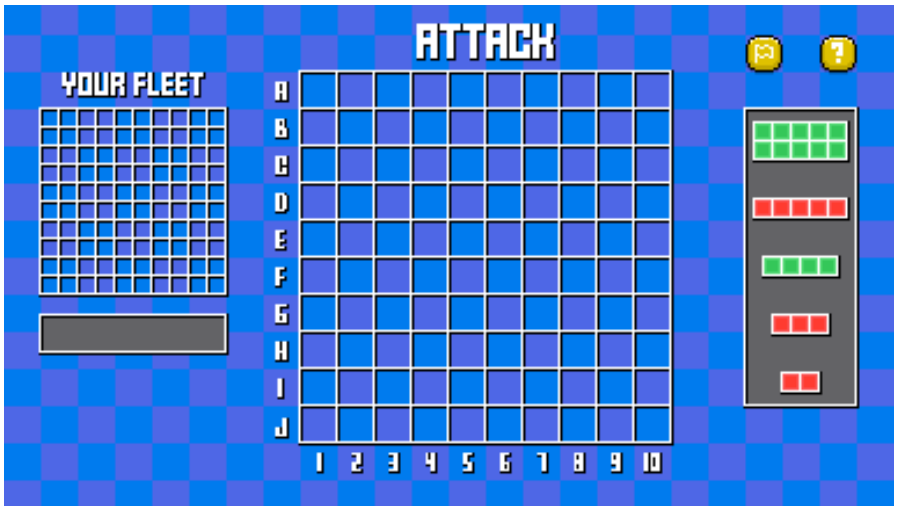


Figure 7: Gameplay Screen



4. Project Management

4.1 Coding Assignments

Table 2: List of Coding Assignments

Screen	User	Entity	GUI
Reid: Screen Manager, & Screen Template classes **Jacob:** Gameplay, Fleet Placement, Difficulty Selection, & Computer Algorithm classes **Logan:** Homepage, Instructions, & Intermediary classes	**Maya**: Combatant & Environment classes **Kaitlyn**: Player class	**Barry:** Grid, Grid Square, & Ship classes **Shirley**: Button, Coordinate, Square Type, & classes	**Shirley:** Sprites **Logan:** Sprites

4.2 Timeline

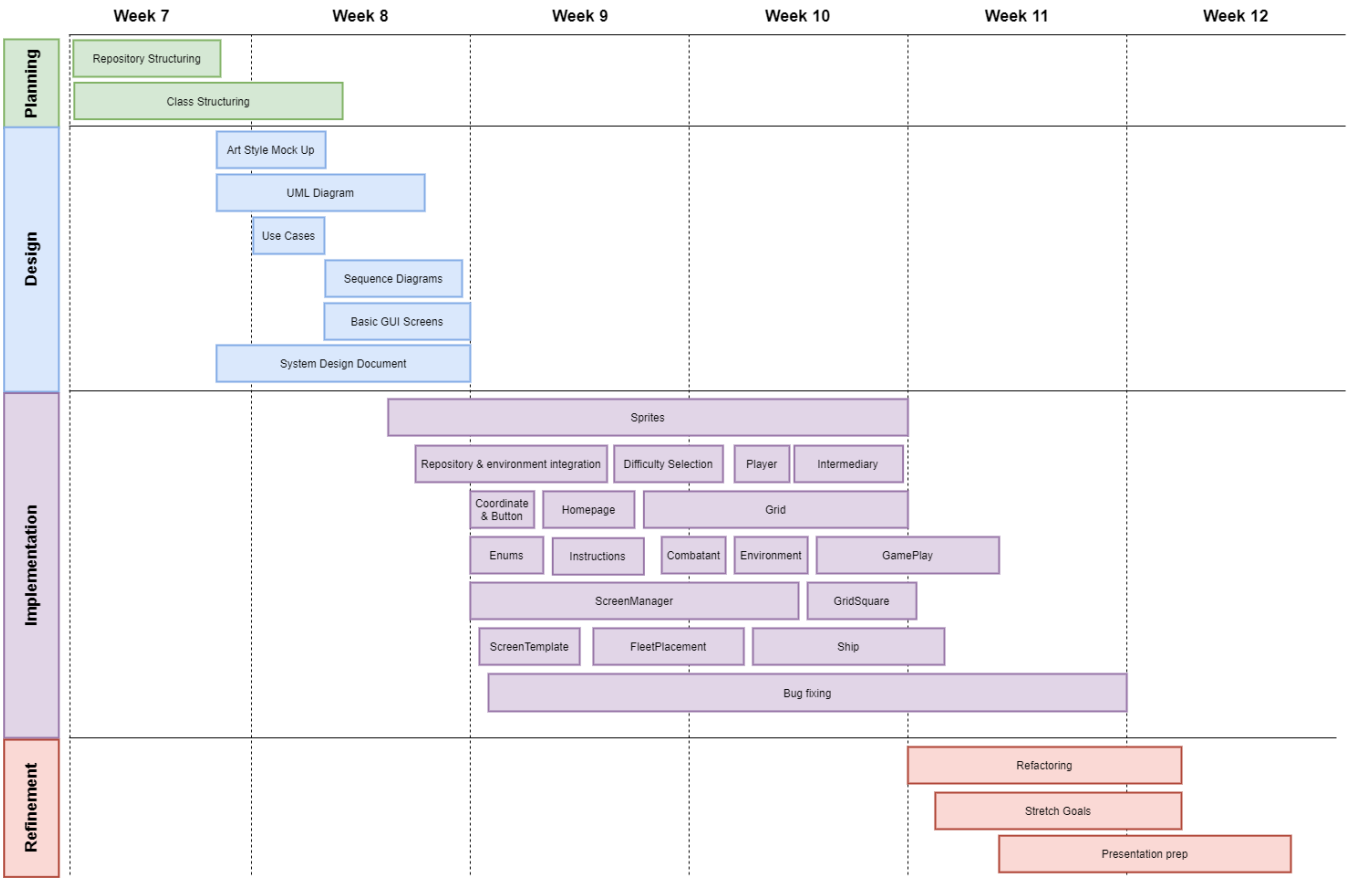



Figure 9: Gantt Chart